

2025 学而思 CSP-J 第二轮认证全真模拟

入门级

题目名称	计算器	战争精英	游戏操作日志	僵尸危机改
题目类型	传统型	传统型	传统型	传统型
目录	calc	peace	game	zombie
可执行文件名	calc	peace	game	zombie
输入文件名	calc.in	peace.in	game.in	zombie.in
输出文件名	calc.out	peace.out	game.out	zombie.out
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	1.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
子任务数目	20	10	20	10
测试点是否等分	是	是	是	是

提交源程序文件名

对于 C++ 语言	calc.cpp	peace.cpp	game.cpp	zombie.cpp
-----------	----------	-----------	----------	------------

编译选项

对于 C++ 语言	-O2 -std=c++14
-----------	----------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 `main()` 的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参考考场的具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈空间内存限制与题目的内存限制一致。
8. 全国统一评测时采用的机器配置为：Inter(R) Core(TM) i7-8700K CPU @3.70GHz，内存 32GB。上述时限以此配置为准。
9. 只提供 Linux 格式附加样例文件。
10. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

计算器 (calc)

【题目描述】

小猴得到了一个特别的计算器，这个计算器只能输入10的幂次方，并且只能进行加法或者乘法。

小猴使用计算器按下了一长串数字，由于数字实在是太长了，因此小猴根本不知道计算机算出的结果是否正确。现在小猴找到了你，希望你帮他算一算结果，方便小猴对比。**【输入格式】**

从文件 *calc.in* 中读入数据。

输入文件一共三行，第一行和第三行分别有一个整数，表示准备计算的数字，保证这两个数字都是10的幂次方。

第二行一个字符，只能是*（乘）或者+（加），表示需要进行哪种运算。

【输出格式】

输出到文件 *calc.out* 中。

输出一行一个整数，表示运算的结果。

【样例 1 输入】

```
1 1000
2 *
3 100
```

【样例 1 输出】

```
1 100000
```

【样例 2 输入】

```
1 1000000000
2 +
3 10
```

【样例 2 输出】

```
1 1000000010
```

【样例 3 输入】

```
1 1
2 *
3 100000
```

【样例 3 输出】

1 100000

【数据范围】

对于30%的数据，数字的长度小于10；

对于60%的数据，数字的长度不超过100；

对于100%的数据，数字的长度不超过 10^5 。

战争精英 (peace)

【题目描述】

小Z是一名忠实的游戏爱好者，最近他迷上了一款叫作《战争精英》的第一人称射击类游戏。在这款游戏中，每一局都会由1000名玩家使用各类枪械相互击杀，直到最终剩下唯一的玩家为止。通过没日没夜的练习，小Z终于成为了该游戏的高手。

作为小Z的小伙伴，小W很好奇小Z究竟有多厉害。某一天小W问小Z："你平均每场能够击杀多少人呀？把你最近的战绩给我看看吧。我会在这些击杀数中去掉一个最高的数字，去掉一个最低的数字，最后将剩下的数字取平均数，从而计算出你的平均击杀数。"

小Z的手机里记录着他最近的连续 N 场比赛的击杀人数，按时间顺序依次是 $A_1, A_2, A_3, \dots, A_N$ ，其中 A_1 表示小Z刚刚结束的那一局的击杀数，而 A_N 表示小Z手机中记录的最早一场的击杀记录。但是小Z并没有把这 N 场的战绩都告诉小W，而是只告诉了小W自己最近 K 场($3 \leq K \leq N$)的战绩，也就是 $A_1, A_2, A_3, \dots, A_K$ ，因为此时小W计算出的平均分是最大的。

聪明的你来帮小Z算一下，究竟要告诉小W最近的多少场游戏的击杀数，才能让小W计算的得到的平均击杀数最大呢？如果有多个不同的 K 都能满足小W计算的平均击杀数最大，小Z会倾向于多给小W看一些战绩，因此小Z会选择这些 K 中最大的。

【输入格式】

从文件 `peace.in` 中读入数据。
第一行一个正整 N ，表示记录了最近 N 场的战绩。
接下来 N 行每行一个正整数，分别表示 $A_1, A_2, A_3, \dots, A_N$ 。

【输出格式】

输出到文件 `peace.out` 中。
输出一个正整数 K 。

【样例 1 输入】

1	5
2	10
3	6
4	7
5	1
6	8

【样例 1 输出】

1	5
---	---

【样例 1 说明】 $K = 3$ 时平均数为7,
 $K = 4$ 时平均数为6.5,

$K = 5$ 时平均数为7,
按照提议小Z会选择平均数最大且 K 尽量大的 $K = 5$ 。

【样例 2 输入】

1	8
2	9
3	5
4	7
5	9
6	2
7	2
8	2
9	9

【样例 2 输出】

1	4
---	---

【样例 3】

见选手目录下的 *peace/peace3.in* 与 *peace/peace3.ans*。

【数据范围】

对于30%的数据， $N \leq 1000$
对于100%的数据， $N \leq 10^6, 0 \leq A_i < 1000$

游戏操作日志 (game)

【题目描述】

小猴在游戏中的平面地图上操作他的角色，角色初始位置的横坐标为 x ，纵坐标为 y 。
游戏的后台日志记录下了小猴的所有操作，分为以下3种：

- (1) 水平移动，用一个字符 x 和一个整数 t 表示。作用是将角色位置的横坐标 x 变为 $x + t$ 。
- (2) 垂直移动，用一个字符 y 和一个整数 t 表示。作用是将角色位置的纵坐标 y 变为 $y + t$ 。
- (3) 交换，用一个字符串 $swap$ 表示，作用是交换 x 和 y 的值。

给出所有 n 个操作，小猴想知道，如果他按顺序执行第 l 到第 r 个操作，他的角色最终位置是什么？输入包含多组询问。

【输入格式】

- 从文件 `game.in` 中读入数据。
- 第 1 行，3 个整数 n, x, y 。
- 第 2 ~ $n + 1$ 行，每行表示一个操作。水平移动用一个字符 x 和一个整数 t 表示；垂直移动用一个字符 y 和一个整数 t 表示；交换用一个字符串 $swap$ 表示。
- 第 $n + 2$ 行，1 个正整数 m 。
- 第 $n + 3 \sim m + n + 2$ 行，每行两个正整数 l, r ，表示询问按顺序执行第 l 到第 r 个操作后角色的最终位置。

【输出格式】

- 输出到文件 `game.out` 中。
- 输出 m 行，对每个询问输出一行，包含两个整数 x, y ，用空格分隔，表示最终位置的横坐标和纵坐标。

【样例 1 输入】

```
1 6 0 0
2 x 1
3 y -3
4 swap
5 x 9
6 y 5
7 x -3
8 3
9 1 2
10 2 4
11 1 6
```

【样例 1 输出】

```
1 1 -3
2 6 0
3 3 6
```

【样例 1 说明】

角色初始在(0,0)处。

依次执行第1 ~ 2个操作，位置变化： $(0,0) \rightarrow (1,0) \rightarrow (1,-3)$ 。

依次执行第2 ~ 4个操作，位置变化： $(0,0) \rightarrow (0,-3) \rightarrow (-3,0) \rightarrow (6,0)$ 。

依次执行第1 ~ 6个操作，位置变化： $(0,0) \rightarrow (1,0) \rightarrow (1,-3) \rightarrow (-3,1) \rightarrow (6,1) \rightarrow (6,6) \rightarrow (3,6)$ 。

【样例 2 输入】

```
1 6 -2 7
2 y 7
3 x -7
4 y 4
5 x 8
6 x -5
7 x 7
8 3
9 5 6
10 1 2
11 1 6
```

【样例 2 输出】

```
1 0 7
2 -9 14
3 1 18
```

【样例 3 输入】

```
1 12 5 7
2 y 23
3 x 70
4 swap
5 x -97
6 y 86
```

```
7 swap
8 y 4
9 x -5
10 swap
11 swap
12 y -20
13 x 20
14 5
15 3 7
16 4 9
17 5 11
18 7 10
19 1 12
```

【样例 3 输出】

```
1 91 -86
2 -88 88
3 88 -11
4 0 11
5 176 -83
```

【数据范围】

对 40% 数据, $n, m \leq 5000$ 。

另外有 20% 数据, 没有 swap 操作。

对 100% 数据, $1 \leq n, m \leq 10^5$, $-10^9 \leq x, y, t \leq 10^9$, $1 \leq l \leq r \leq n$ 。

僵尸危机改 (zombie)

【题目描述】

小猴是一名忠实的游戏爱好者，最近他迷上了一款叫作《僵尸危机》的双人益智类小游戏，于是叫上了小W一起玩。小猴和小W被困在了一个迷宫里的两个不同角落，在这个迷宫里有僵尸，僵尸会杀人，并且会不断繁殖自己，小猴和小W如果能在被僵尸抓住前汇合，他们就可以获得游戏的胜利。

小猴和小W都可以沿着上下左右四个方向进行移动，每移动一格算作一步。小猴每秒可以移动三步，小W每秒可以移动一步。每秒每个僵尸都将分裂成很多个部分去占领距离它们两步以内（包含两步）的格子，直到它们占满整个迷宫，假设每秒都是僵尸先分裂，然后接着小猴和小W才进行移动，如果他们两个中任何一个人到达有僵尸的格子，游戏就失败了。

注：新产生的僵尸是和之前僵尸一样的，也能持续扩张。

【输入格式】

从文件 `zombie.in` 中读入数据。

第一行一个整数 T 代表测试用例个数

第二行两个整数 n, m 表示迷宫的尺寸

接下来 n 行描述迷宫，每行包含 m 个字母，字母可能是

. 表示一个空地，人和僵尸都可以走；

x 表示一个墙，只有人不可以走，僵尸可以到；

M 表示小猴；

G 表示小W；

Z 表示僵尸。

保证输入的迷宫数据中，只包含一个字母 M，一个字母 G和 最多两个 字母 Z。

【输出格式】

输出到文件 `zombie.out` 中。

输出 T 行整数，对于每组数据，如果小猴和小 W 能成功见面的话，输出最少时间，否则，输出-1。

【样例 1 输入】

```
1 3
2 5 6
3 XXXXXX
4 XZ..ZX
5 XXXXXX
6 M.G...
7 .....
8 7 6
9 XXXXXX
10 XZZ..X
11 XXXXXX
12 .....
13 .....
14 .....
15 M....G
16 10 10
17 .....
18 ..X.....
19 ..M.X...X.
20 X.....
21 .X..X.X.X.
22 .....X
23 ..XX....X.
24 X....G...X
25 ...ZX.X...
26 ...Z..X..X
```

【样例 1 输出】

```
1 1
2 2
3 -1
```

【样例 1 说明】

第一个测试用例：Z到M，G的距离都大于2，一秒内Z碰不到M和G，M与G可以在一秒内遇到；

第二个测试用例：M和G在2秒内就可相遇。

第三个测试用例：Z在两秒内会追上G，M在两秒内碰不到G，所以输出-1

【样例 2】

见选手目录下的 *zombie/zombie2.in* 与 *zombie/zombie2.ans*。

【样例 3】

见选手目录下的 *zombie/zombie3.in* 与 *zombie/zombie3.ans*。

【数据范围】

共有 10 个数据点：

数据点 1、2：输入中没有字符 Z；

数据点 3、4：每个迷宫只有第一行第一列是字符 Z，其他位置都不是 Z；

数据点 5~10：无特殊性质。

对于100%的数据， $1 < n, m \leq 800$ ， $1 \leq T \leq 5$ 。